

```
factorial(5)
  ↓
return factorial(5 - 1, 1 * 5)
  ↓
return factorial(4 - 1, 5 * 4)
  ↓
return factorial(3 - 1, 20 * 3)
  ↓
return factorial(2 - 1, 60 * 2)
  ↓
return 120
```

Figure 8-1: The process of transformations that `factorial(5)` makes to the integer 120

Rearranging your recursive calls as the last action in the function and adding accumulators can make your code even harder to understand than typical recursive code. But that's not the only downside of tail recursion, as we'll see in the next section.